Wormhole Addresses: Recursive Zero-Knowledge Private Settlement for Post-Quantum Blockchains

Christopher Smith Ethan Cemer

March 2026

Abstract

Post-quantum signature schemes impose large public keys and signatures, increasing on-chain costs while still leaking transaction structure. We present *Quantus Wormhole*: users deposit to addresses derived from secrets, then prove in zero knowledge that (i) the deposit exists in authenticated state, (ii) the block-header commitment is valid, (iii) the nullifier is fresh, and (iv) exits satisfy fee constraints. Recursive aggregation amortizes verification across many withdrawals. The first aggregation layer pads batches with dummy proofs and shuffles locally to enlarge the explanation class for balance-linking adversaries. We formalize the leaf relation, analyze layer-0 privacy, prove deposit binding and one-time withdrawal, and report proving times for batch sizes 2–49.

1 Introduction

Shor's algorithm breaks elliptic-curve signatures [15]. Post-quantum replacements like ML-DSA have much larger keys and signatures, increasing bandwidth, state growth, and verification cost [7]. Simply swapping signature schemes sacrifices throughput and does nothing for privacy.

We take a different approach: instead of authorizing spends directly on-chain, users deposit to addresses derived from secrets and later prove in zero knowledge that (i) the deposit exists, (ii) they know the secret, (iii) it hasn't been spent, and (iv) exits satisfy fee rules. Recursive aggregation then compresses many withdrawals into fewer on-chain verifications.

This design builds on EIP-7503's wormhole concept [6], adapted to the Quantus stack with storage-proof-based implementation [13].

Contributions.

1. Formalize the *wormhole-address* spending model: deposits flow to deterministic unspendable addresses derived from a secret; withdrawals require proving knowledge of that secret plus authenticated state inclusion.

2. Specify the leaf relation: public outputs, private witnesses, and sub-circuit constraints for nullifier derivation, unspendable-account derivation, storage-proof verification, fee-constrained dual exits, and block-header commitment.

3. Describe the recursive aggregation architecture. The first aggregation layer batches leaf proofs sharing the same `block_hash`, pads with dummy leaves, shuffles client-locally, and wraps with a circuit forwarding nullifiers and grouped exits.

4. Provide combinatorial privacy analysis of layer-0, showing how dummy padding, hidden batch order, and grouped exits enlarge the explanation class for balance-linking adversaries.

5. Explain why the public commitment covers the full block header (not just the state root), letting the runtime validate that the witnessed state root belongs to canonical history.

6. Report measured build, prove, and verify times using `qp-plonky2` with Poseidon2 recursion, for batch sizes 2–49 (layer-0) and fan-ins 2–32 (layer-1).

7. State security claims for deposit binding, wrapper soundness, one-time withdrawal, spend-path exclusivity, and value conservation, separating proof-level from runtime-level assumptions.

2 Background and Design Goals

2.1 From post-quantum signatures to proof-based settlement

Post-quantum signatures are expensive to publish and verify. Rather than publishing a large signature per spend, users can prove knowledge of a short secret checked against authenticated state. The chain verifies a proof and updates a nullifier set.

2.2 Relationship to wormholes and privacy pools

EIP-7503 proposes *zero-knowledge wormholes*: deposit to secret-derived addresses, then prove knowledge of the secret to withdraw [6]. Quantus differs in three ways: (1) post-quantum-friendly proving, (2) implemented against authenticated state (not just a proposal), and (3) recursive aggregation with dummy padding.

Related systems include privacy pools and shielded ledgers like Zcash and Tornado Cash [3, 5, 6, 10]. Unlike fully shielded ledgers, Quantus proves against native chain state without a separate note commitment tree.

2.3 Why transparent recursion

We use `qp-plonky2`, combining PLONK arithmetization with FRI commitments for recursive verification without trusted setup [1, 9, 14]. Poseidon2 provides efficient in-circuit hashing over prime fields [4].

2.4 Design goals

The protocol aims to satisfy four simultaneous goals:

1. **Scalable authorization.** On-chain work should scale with proof verification and forwarded public outputs, not with raw post-quantum signature material per transaction.

2. **Authenticated settlement.** Every successful withdrawal must be bound to a real deposit recorded in authenticated chain state.

3. **Replay prevention.** A deposit should not authorize multiple withdrawals; nullifiers must uniquely bind proofs to deposits.

4. **Meaningful privacy.** The chain should reveal only the public information needed for settlement: exits, amounts, shared fee parameters, nullifiers, and a block commitment. Funding accounts, secrets, and raw storage proofs should remain private.

3 System, Adversary, and Trust Model

3.1 Roles

The system comprises the following roles.

- **User / wallet.** Generates a secret, derives an wormhole address, funds it, collects the witness data for withdrawal, and may optionally perform first-layer aggregation locally.
- **Proof source or indexer.** Supplies the user with the relevant transfer record, block-header data, and storage proof needed to construct a witness. This role is not trusted for correctness because incorrect data causes proof failure.
- **First-layer aggregator.** A privacy-sensitive local actor that fills a batch with real and dummy leaf proofs, shuffles them, and produces the first recursive wrapper.
- **Higher-layer aggregator.** A potentially delegated actor that composes already aggregated proofs and forwards their public outputs.
- **Runtime / verifier.** Checks block consistency, fee parameters, nullifier freshness, and proof validity before applying settlement.

3.2 State objects

Let $\mathcal{A}$ be the set of 32-byte account identifiers and let σ denote chain state. A withdrawal witness references the following objects:

- an asset identifier a,
- an input amount v_{in}, quantized to the circuit's 0.01-unit resolution,
- a transfer counter c,
- a funding account α_{fund},
- an unspendable deposit account α_{wh},
- a processed storage proof authenticating the transfer record under a state root rt, and
- structured block-header fields ($\mathsf{parent_hash}, b, \mathsf{rt}, \mathsf{extrinsics_root}, \mathsf{digest}$).

The runtime additionally maintains a nullifier set $\mathcal{N}$ of all accepted nullifiers.

3.3 Adversarial capabilities

We assume a probabilistic polynomial-time adversary that can observe the full chain, submit arbitrary deposits, inspect public proof outputs, perform timing analysis, and attempt to replay or malform proof submissions. The adversary does *not* break the collision resistance of the hash function, the knowledge soundness of the proof system, or the chain's block-history availability assumptions.

3.4 Trust assumptions

The protocol relies on the following assumptions.

1. **Hash security.** The Poseidon2 instantiation used by the system remains collision resistant and preimage resistant for the relevant security level [4].
2. **Proof soundness.** The configured Plonky2 recursion stack remains knowledge sound under its standard assumptions [1, 9].
3. **Authenticated history.** The runtime can check that the public block commitment and block number correspond to a valid block in chain history.
4. **Witness availability.** Honest users can obtain the storage proof and block-header data needed to build a witness for their deposit.

4 Protocol Overview

The protocol consists of five stages.

1. Secret generation and deposit. The user samples a secret s and derives an wormhole address

$$\mathsf{WA}(s) = H(H(\text{“}\mathsf{wormhole}'' \parallel s)).$$

The user then funds this address through an ordinary transfer on Quantus. The deposit is accompanied by a chain record that can later be authenticated inside the circuit.

2. Witness acquisition. To withdraw, the user gathers the transfer record, a processed storage proof, the state root to which that proof authenticates, and the structured block-header fields needed to recompute the public block commitment. These data form the private witness except for the public block fields that will be exposed by the proof.

A subtlety is that the protocol does *not* expose the state root directly as its public chain anchor. An earlier design considered doing exactly that, but a bare public state root is insufficient for runtime validation: seeing only a root does not tell the verifier whether that root belongs to any valid canonical block. The implemented design therefore witnesses the structured header privately and exposes a public Poseidon2 commitment to that full header together with the public block number. The runtime validates the committed header against chain history, while the circuit binds the privately used state root in the storage proof to that same committed header.

3. Leaf proof generation. The user chooses up to two exit outputs, computes a nullifier

$$\mathsf{Null}(s, c) = H(H(\text{“}\tilde{}\mathsf{nullif}\tilde{}'' \parallel s \parallel c)),$$

and produces a leaf proof. The proof reveals the nullifier, the exits, the output amounts, the asset identifier, the fee basis points, the public block commitment, and the block number, while keeping the secret, funding account, storage proof, and transfer-specific raw details private.

4. First-layer aggregation. A privacy-sensitive local aggregator batches leaf proofs that share the same public `block_hash`. If the batch is not full, the aggregator pads it with dummy leaves, shuffles the order, recursively merges the proofs, and then applies a wrapper circuit that enforces consistency and forwards grouped exits and nullifiers.

Algorithm 1 High-level withdrawal pipeline

procedure WithdrawAndSettle$(s, c, \alpha_1, \alpha_2, v_1, v_2)$
  derive $\alpha_{\text{wh}} \leftarrow \mathsf{WA}(s)$
  fetch transfer record, block-header fields, and storage proof for counter c
  compute $n \leftarrow \mathsf{Null}(s, c)$
  form public inputs $x = (a, v_1, v_2, f, n, \alpha_1, \alpha_2, h_B, b)$
  form witness w from secret, transfer record, and authenticated state data
  produce leaf proof $\pi \leftarrow \mathsf{Prove}(x, w)$
  optionally aggregate π with same-block leaf proofs
  submit proof and public inputs for runtime verification
end procedure

Field	Type	Meaning
`asset_id`	`u32`	Token identifier; `0` denotes the native asset.
`output_amount_1`	`u32`	Primary output amount after quantization.
`output_amount_2`	`u32`	Secondary output amount; zero if unused.
`volume_fee_bps`	`u32`	Runtime fee parameter in basis points.
`nullifier`	4 felts	Public replay-prevention tag derived from secret and transfer counter.
`exit_account_1`	4 felts	Primary destination account.
`exit_account_2`	4 felts	Secondary destination account or all zeros.
`block_hash`	4 felts	Public Poseidon2 commitment to structured block-header fields.
`block_number`	`u32`	Height of the block whose header is committed by `block_hash`.

Table 1: Logical leaf public inputs. In the current implementation this logical interface occupies 21 field elements.

5. Settlement. The runtime verifies the aggregate proof, checks that the public `block_hash` matches the referenced block number, verifies that the fee parameter matches runtime configuration, rejects reused nullifiers, and settles the exits.

5 Formal Specification and Interfaces

5.1 Leaf public inputs

The implemented leaf proof exposes exactly nine logical public fields. In the repository layout, these occupy 21 field elements because the nullifier, each exit account, and the block hash each consume four field elements [13].

5.2 Leaf private witness

The witness consists of the user's secret, the transfer counter, the funding account, the derived unspendable account, the processed storage proof, the state root, the parent hash, the extrinsics root, the encoded digest logs, and the input amount recorded in storage.

5.3 Leaf relation

Let $x = (a, v_1, v_2, f, n, \alpha_1, \alpha_2, h_B, b)$ denote the public input tuple and let w denote the witness described above. The leaf relation $\mathcal{R}_{\text{leaf}}$ holds when there exists a witness w such that:

Field	Type	Meaning
`secret`	32 bytes	User secret from which both nullifier and wormhole address are derived.
`transfer_count`	`u64`	Per-transfer counter used in nullifier derivation.
`funding_account`	4 felts	Source account associated with the transfer record.
`unspendable_account`	4 felts	Derived wormhole address $\mathsf{WA}(s)$.
`storage_proof`	structured witness	Processed inclusion proof for the transfer record under the state root.
`state_root`	4 felts	Root authenticating the storage proof.
`parent_hash`	4 felts	Parent block hash used in header recomputation.
`extrinsics_root`	4 felts	Extrinsics root from the block header.
`digest`	fixed bytes	Encoded digest logs.
`input_amount`	`u32`-compatible quantity	Quantized deposited amount before fee deduction.

Table 2: Private witness components used by the leaf proof.

(C1) **Nullifier correctness.** $n = \mathsf{Null}(s, c)$.

(C2) **Entangled-address correctness.** $\alpha_{\text{wh}} = \mathsf{WA}(s)$.

(C3) **Authenticated inclusion.** The processed storage proof verifies that a transfer record keyed by the relevant funding account, wormhole account, and counter exists under rt.

(C4) **Header binding.** The structured block-header fields hash to the public commitment h_B, and the public block number equals the header's block number.

(C5) **Fee constraint.** The disclosed exits satisfy

$$(v_1 + v_2) \cdot 10000 \leq v_{\text{in}} \cdot (10000 - f).$$

(C6) **Range constraints.** The public quantities lie within the ranges expected by the runtime interface.

Definition 1 (Leaf relation)**.** *The relation $\mathcal{R}_{\text{leaf}}$ is the set of all (x, w) satisfying conditions (C1)–(C6).*

5.4 Runtime interface

The proof system does not, by itself, settle a withdrawal. The runtime must also check the following: (i) the referenced block exists in chain history, (ii) the public fee parameter matches the active runtime fee configuration, (iii) the public nullifier has not appeared before, and (iv) any asset-specific accounting rules are satisfied. These external checks are part of the overall security argument.

6 Recursive Aggregation Architecture

6.1 Leaf layout and wrapper output layout

Each leaf proof contributes 21 public-input field elements. The first aggregation layer produces a wrapper proof whose semantic public output consists of:

- a fixed 8-felt header containing the number of exit slots, asset identifier, fee basis points, block hash, and block number;

- $2N$ exit slots, each represented as a 1-felt amount plus a 4-felt exit account; and
- N nullifiers, each occupying 4 felts.

Thus the meaningful forwarded payload is $14N + 8$ felts, although the current wrapper pads its public-input vector out to $21N + 8$ felts for layout regularity.

6.2 First-layer policy

The first aggregation layer is privacy-sensitive. A valid batch must satisfy:

1. Every real leaf proof in the batch shares the same public `block_hash`.
2. Every real leaf proof shares the same `asset_id` and `volume_fee_bps`.
3. If fewer than N real leaves are available, the batch is padded with dummy leaves.
4. The leaf order is shuffled before wrapping, subject to implementation-specific handling that preserves at least one real leaf in a stable position for correctness.
5. Duplicate exit accounts are grouped so that identical destinations accumulate a summed amount in one slot while duplicate slots are zeroed.
6. All nullifiers are forwarded, with dummy nullifiers replaced by fresh random values.

Because the padding and shuffle are part of the privacy mechanism, layer-0 should be executed locally by the wallet whenever possible; a delegated layer-0 service would observe the client's raw child proofs before they are hidden inside the wrapper and would therefore collapse much of the ambiguity quantified in section 7.8.

6.3 Wrapper logic

At a high level, the wrapper circuit performs four tasks.

1. It detects dummy leaves by checking whether the child `block_hash` is all zeros.
2. It enforces that every non-dummy child agrees with the reference child's `block_hash`, `asset_id`, and `volume_fee_bps`.
3. It computes grouped exit slots by summing amounts for identical exit accounts and zeroing duplicate slots.
4. It forwards real nullifiers and substitutes random dummy nullifiers for dummy children.

6.4 Higher-layer aggregation

Higher recursive layers operate over already aggregated proofs rather than raw leaf witnesses. This separation is important: the privacy-critical shuffle and dummy-handling logic stays local to the first layer, while later layers can be delegated because they receive opaque layer-0 aggregates rather than the client's raw proof set. In particular, a layer-1 aggregator learns only the public output of layer-0 and therefore does not observe the client's internal real-versus-dummy placement or pre-grouped exit set.

Claim	Additional assumptions outside the circuit relation
Leaf-to-deposit binding	Runtime validates that the public `block_hash`/`block_number` pair denotes a real block and that the processed storage proof format matches the authenticated state representation.
First-layer wrapper soundness	The recursive verifier gadgets correctly enforce child-proof validity, and the wrapper implementation matches its stated grouping and forwarding logic.
One-time withdrawal per authenticated deposit record	Runtime rejects reused nullifiers, including duplicates within one aggregate submission; the authenticated transfer record fixes the transfer counter used in nullifier derivation.
Spend-path exclusivity for wormhole addresses	The surrounding chain only admits direct spends from addresses backed by valid external credentials, and no efficient adversary can manufacture such credentials for an wormhole address.
Global value conservation	Runtime settles exactly the forwarded public outputs and records the associated nullifiers.

Table 3: Where the security argument relies on runtime or chain-interface assumptions beyond circuit correctness.

7 Security and Privacy Analysis

This section separates *proof-level* statements from *system-level* statements. Proof-level statements concern the relations enforced by the leaf circuit and the first-layer wrapper. System-level statements additionally rely on runtime checks outside the proof system: the public `block_hash`/`block_number` pair must refer to a real block in chain history, the public fee parameter must match runtime configuration, nullifiers must be rejected on reuse (including duplicates inside one candidate batch), and settlement must apply exactly the forwarded exit amounts. Appendix A expands the proof sketches below into code-aware reductions tied to the current dummy-template and artifact-loading paths.

7.1 Authenticated deposit records

To tighten the security argument, it is useful to name the object that the proof is actually certifying.

Definition 2 (Authenticated deposit record)**.** *An* authenticated deposit record *is a tuple*

$$d = (a, v_{\text{in}}, c, \alpha_{\text{fund}}, \alpha_{\text{wh}}, h_B, b)$$

for which there exists witness data w such that $(x, w) \in \mathcal{R}_{\text{leaf}}$, where x exposes asset identifier a, block commitment h_B, block number b, and exits satisfying the fee constraint against v_{in}. Intuitively, d is a deposit object that has been authenticated by the storage-proof and header-binding subcircuits described in section 5 and **??**.

A *real* leaf is a leaf whose public `block_hash` is nonzero. A *dummy* leaf is a padding object identified by the sentinel `block_hash` $= 0^{32}$ and used only inside first-layer aggregation.

7.2 Completeness

Theorem 1 (Leaf completeness)**.** *If an honest prover holds witness data w for which $(x, w) \in \mathcal{R}_{\text{leaf}}$, then the prover can generate a leaf proof accepted by the leaf verifier.*

Proof sketch. Completeness follows from the correctness of the individual circuit fragments and of the proving system. Each fragment enforces a deterministic relation between its witness variables and public outputs. When the nullifier, wormhole address, storage proof, header commitment, fee inequality, and range constraints are all satisfied, the assembled witness satisfies $\mathcal{R}_{\text{leaf}}$, so the prover can produce an accepting proof. □

Theorem 2 (Wrapper completeness)**.** *Suppose a first-layer aggregator is given a full batch of valid child proofs, padding any missing positions with the designated dummy template. If all real children share the same public* `block_hash`*,* `asset_id`*, and* `volume_fee_bps`*, then the aggregator can produce an accepting first-layer wrapper proof.*

Proof sketch. The wrapper verifies the child proofs recursively, checks equality of the shared batch parameters across all non-dummy children, applies the implemented exit-grouping logic, and forwards nullifiers together with dummy replacements. When the inputs satisfy those conditions, the wrapper constraints are satisfiable by construction. □

7.3 Leaf soundness and deposit binding

Theorem 3 (Leaf soundness)**.** *Assume the leaf proof system is knowledge sound. For every accepted leaf proof (x, π), there exists an extractor that outputs a witness w such that $(x, w) \in \mathcal{R}_{\text{leaf}}$.*

Proof sketch. This is the standard consequence of knowledge soundness for the leaf relation. The extractor outputs values for the secret, counter, funding account, wormhole account, storage-proof witness, header fields, and input amount that satisfy conditions (C1)–(C6) from section 5. No further claim is made here about whether the committed block actually exists in canonical chain history; that is deferred to the runtime check on the public `block_hash`/`block_number` pair. □

Corollary 4 (Authenticated-deposit binding)**.** *Assume leaf soundness and correct runtime validation of the public block commitment. Then every accepted real leaf proof binds to an authenticated deposit record and certifies that its disclosed exits satisfy the protocol fee rule for that record.*

Proof. By leaf soundness, an accepting real leaf yields a witness satisfying $\mathcal{R}_{\text{leaf}}$. Conditions (C3) and (C4) bind the proof to an authenticated transfer record under the state root committed by the structured header fields, while condition (C5) enforces the fee inequality. Runtime validation upgrades the public header commitment from a merely self-consistent header witness to a real block in chain history. □

7.4 First-layer wrapper soundness

Theorem 5 (First-layer wrapper soundness)**.** *Assume the recursive verifier gadgets and the wrapper circuit are sound. Let Y be the public output of an accepted first-layer wrapper proof of arity N. Then there exists a list of child instances*

$$((x_1, \pi_1, \delta_1), \ldots, (x_N, \pi_N, \delta_N)),$$

where $\delta_i \in \{0, 1\}$ indicates whether child i is dummy, such that:

(W1) if $\delta_i = 0$, then π_i is an accepted leaf proof on public input x_i;

(W2) if $\delta_i = 1$, then child i is recognized by the dummy sentinel `block_hash` $= 0^{32}$*;*

(W3) all non-dummy children share the same public header tuple exposed by Y: a common block hash, asset identifier, fee parameter, and block number;

(W4) the exit slots exposed by Y are exactly the result of applying the wrapper's grouping rule to the multiset of non-dummy child exits; and

(W5) the nullifier region of Y contains every real child nullifier, while dummy positions are filled by wrapper-supplied replacement values.

Proof sketch. Soundness of the recursive verifier gadgets ensures that every accepted child proof referenced by the wrapper witness is itself valid. The wrapper then recomputes its public outputs from those children subject to explicit conditional checks: non-dummy children must agree on the batch `block_hash`, `asset_id`, and `volume_fee_bps`; exit slots are formed by the implemented grouping-and-zeroing logic; and nullifiers are forwarded child-by-child with dummy substitution in positions marked by the sentinel. The shared `block_number` claim is obtained by combining wrapper soundness with leaf soundness: each real child is itself a sound leaf proof satisfying condition (C4), so a common `block_hash` implies a common committed header and hence a common `block_number`. A line-by-line version of this argument, together with the exact code-path linkage for dummy padding, is given in section A. Hence any accepting wrapper proof can be explained by a concrete list of real and dummy children satisfying (W1)–(W5). □

Lemma 6 (Dummy-padding safety)**.** *Assume the designated dummy template exposes zero-valued exits, and the runtime only settles the public exits produced by the wrapper. Then dummy padding cannot increase settled value. Random replacement of dummy nullifiers affects only accidental-collision probability and therefore liveness, not soundness.*

Proof. Under the stated dummy-template assumption, a dummy child contributes no positive exit amount to the grouped wrapper outputs. Replacing its nullifier with a fresh filler does not create an authenticated deposit record or any additional settled value; at worst, an accidental nullifier collision causes rejection of the aggregate. Thus dummy padding cannot authorize value that was absent from the real children. □

7.5 One-time withdrawal and double-spend resistance

Theorem 7 (One-time withdrawal per authenticated deposit record)**.** *Assume:*

(D1) the runtime rejects reused nullifiers, including duplicates within one candidate aggregate submission;

(D2) the authenticated transfer record fixes the transfer counter c used in nullifier derivation; and

(D3) the hash function used in $\mathsf{WA}(\cdot)$ and $\mathsf{Null}(\cdot,\cdot)$ is collision resistant and second-preimage resistant for the relevant domain-separated inputs.

Then no probabilistic polynomial-time adversary can obtain two accepted real withdrawals from the same authenticated deposit record except with negligible probability.

Proof sketch. Suppose two accepted real withdrawals correspond to the same authenticated deposit record. By authenticated-deposit binding, extraction yields witnesses (s, c) and (s', c') together with the same authenticated record

$$(a, v_{\mathrm{in}}, c, \alpha_{\mathrm{fund}}, \alpha_{\mathrm{wh}}, h_B, b).$$

Because the record fixes the counter, we have $c = c'$. Because the record also fixes the wormhole address, both witnesses satisfy

$$\mathsf{WA}(s) = \alpha_{\mathrm{wh}} = \mathsf{WA}(s').$$

If $s \neq s'$, the adversary has found a collision or second preimage for the wormhole-address derivation. Therefore $s = s'$ except with negligible probability. But then

$$\mathsf{Null}(s, c) = \mathsf{Null}(s', c'),$$

so both withdrawals expose the same nullifier. The runtime rejects the second occurrence by assumption (D1). Hence one authenticated deposit record can underwrite at most one accepted real withdrawal. □

The theorem above is intentionally phrased in terms of an *authenticated deposit record*, not an informal notion of "the same funds." If a future protocol version supports explicit partial-withdrawal state transitions, that stronger state model should be proved separately rather than inferred from the present one-shot nullifier discipline.

7.6 Operational unspendability and spend-path exclusivity

The protocol needs one assumption that is partly external to the circuit: funds sent to an wormhole address should not also be directly spendable through an ordinary account-authentication path.

Proposition 8 (Spend-path exclusivity under the chain's address discipline)**.** *Let* $\mathsf{Addr} : \mathcal{PK}_\Sigma \to \{0,1\}^{256}$ *denote the chain's ordinary address derivation from externally controlled public keys. Suppose:*

(U1) every direct spend accepted by the base chain must be authorized by a valid public key $pk \in \mathcal{PK}_\Sigma$ *whose address is* $\mathsf{Addr}(pk)$*; and*

(U2) no efficient adversary can find $pk \in \mathcal{PK}_\Sigma$ *such that* $\mathsf{Addr}(pk) = \mathsf{WA}(s)$ *for a target wormhole address, except by breaking the collision or preimage resistance of the derivation.*

Then value sent to $\mathsf{WA}(s)$ *can leave the system only through the proof-authorized settlement path.*

Proof sketch. Any direct spend must use some valid external credential pk by assumption (U1). If such a spend exists from $\mathsf{WA}(s)$, then $\mathsf{Addr}(pk) = \mathsf{WA}(s)$, contradicting (U2) except with negligible probability. Therefore the only remaining spend path is the proof-authorized one. We treat spend-path exclusivity as a system assumption about the surrounding chain interface rather than as a theorem of the circuit in isolation. □

7.7 Value conservation

Lemma 9 (Per-leaf fee bound)**.** *For every accepted real leaf proof, the sum of its two public output amounts is at most the authenticated input amount net of the public fee parameter.*

Proof. This is exactly condition (C5) of $\mathcal{R}_{\mathrm{leaf}}$, obtained by extraction from leaf soundness. □

Lemma 10 (Wrapper amount preservation)**.** *Assume the first-layer wrapper groups identical exit accounts by summing their amounts into one slot and zeroing duplicate slots. Then the sum of the forwarded exit amounts equals the sum of the exit amounts of the real child proofs.*

Proof. The wrapper's grouping step is a partition of the multiset of real child exits by destination account. Replacing each partition class by a single slot whose amount is the class sum preserves the total amount. Zeroing duplicate slots removes representation duplicates, not value. □

Theorem 11 (Global value conservation)**.** *Assume:*

(V1) leaf soundness,

(V2) first-layer wrapper soundness,

(V3) runtime rejection of reused nullifiers, and

(V4) runtime settlement that applies exactly the forwarded public exit amounts.

Then the total value settled by any accepted first-layer aggregate proof is at most the total value of its underlying authenticated deposit records after protocol fees. Across multiple accepted proofs, no authenticated deposit record is counted more than once.

Proof sketch. By first-layer wrapper soundness, an accepted aggregate expands to a multiset of real child leaves and dummy padding. By dummy-padding safety, dummies contribute no settled value. By authenticated-deposit binding and the per-leaf fee bound, each real child contributes at most its authenticated deposit amount net of fees. By wrapper amount preservation, the sum of forwarded exit amounts equals the sum of the real child exit amounts. Finally, one-time withdrawal per authenticated deposit record rules out counting the same authenticated record twice via multiple accepted nullifiers. Hence settled value never exceeds the aggregate authenticated input value net of fees. □

7.8 Layer-0 explanation classes and balance-linking difficulty

The first aggregation layer does not hide the final exits or their amounts, but it does make leaf-level linkage more ambiguous. The key point is that the layer-0 wrapper is many-to-one: multiple ordered batches of real and dummy leaves can induce the same public aggregate once dummy nullifiers are replaced and duplicate exits are grouped.

Definition 3 (Layer-0 explanation class)**.** *Fix a first-layer public output Y. The* explanation class $\mathcal{E}(Y)$ *is the set of ordered batches of real leaves and dummy leaves, together with per-leaf two-output assignments, that are consistent with the leaf fee rule and that produce Y under the implemented layer-0 padding, shuffle, grouping, and dummy-nullifier-substitution logic.*

A balance-linking adversary who sees only Y must distinguish among the elements of $\mathcal{E}(Y)$. By contrast, a delegated layer-0 aggregator that receives the client's raw leaf proofs before padding and shuffling learns one concrete preimage of Y and therefore collapses this ambiguity. This is the formal reason that layer-0 is privacy-sensitive and should be client-local.

Proposition 12 (Client-local layer-0 dominates delegated layer-0 for linkage privacy)**.** *Fix any first-layer public output Y. A downstream observer when layer-0 is performed client-locally receives only Y, whereas a delegated layer-0 aggregator receives both Y and one specific preimage $B \in \mathcal{E}(Y)$. Under a uniform prior over $\mathcal{E}(Y)$, the optimal exact leaf-to-output linkage success probability is $1/|\mathcal{E}(Y)|$ for the former view and 1 for the latter.*

Proof. The downstream observer receives only Y, so any exact linkage guess is a guess of which element of $\mathcal{E}(Y)$ generated the aggregate. Under the stated prior, the optimal strategy succeeds with probability $1/|\mathcal{E}(Y)|$. A delegated layer-0 aggregator, however, observes the raw child proofs before padding and shuffling, so it knows the realized preimage B directly. □

The size of $\mathcal{E}(Y)$ grows combinatorially whenever multiple hidden child outputs collapse into one grouped exit.

Proposition 13 (Combinatorial lower bound from grouped exits)**.** *Let $\Delta = 0.01$ be the circuit quantization unit. Fix a grouped exit account a in Y whose public forwarded amount is $S_a \in \Delta\mathbb{Z}_{>0}$. If that total arose from $r_a \geq 1$ positive real child outputs to the same account, then the number of positive quantized decompositions of S_a into r_a hidden child contributions is*

$$\binom{S_a/\Delta - 1}{r_a - 1}.$$

Consequently, if A^+ denotes the set of grouped exit accounts that received contributions from more than one real child output, then

$$|\mathcal{E}(Y)| \geq \prod_{a \in A^+} \binom{S_a/\Delta - 1}{r_a - 1},$$

with additional multiplicative ambiguity coming from hidden dummy placement, private batch order, and the assignment of at most two outputs to each real leaf.

Proof. For a fixed account a, write $S_a = m_a\Delta$ with $m_a \in \mathbb{Z}_{>0}$. Counting positive integer solutions to

$$x_1 + \cdots + x_{r_a} = m_a$$

gives the classical stars-and-bars count $\binom{m_a-1}{r_a-1}$. Each such solution corresponds to a distinct quantized decomposition of the grouped amount into hidden child outputs. Independent grouped accounts multiply, yielding the stated lower bound. Any uncertainty about which leaves were dummy, how the client ordered the batch before wrapping, or which of a leaf's two output positions carried a given contribution can only enlarge the explanation class. □

The implementation preserves one real child in slot 0. This reduces but does not eliminate occupancy ambiguity from shuffling.

7.9 Privacy boundary

Two privacy aspects:

Proof-level witness hiding: Secrets, funding accounts, storage paths, and header details are not public inputs. Leakage would require breaking the proof system or side channels.

System-level privacy: Output amounts, exit accounts, nullifiers, and block references are public. Layer-0 padding enlarges the explanation class, but this is not full unlinkability—amount leakage, address reuse, and timing correlation remain limitations.

8 Implementation

The codebase is organized as separate crates [13]:

8.1 qp-plonky2 and Poseidon2

Wormhole uses `qp-plonky2`, our Plonky2 fork with Poseidon2 support integrated into the recursive FRI pipeline [14]. When development began, Plonky3 lacked the recursion surface we needed [11]; Plonky3 has since added recursion support [12], but our benchmarks use the deployed `qp-plonky2` stack.

Crate	Role
`qp-wormhole-inputs`	Shared public-input and aggregated-output data structures.
`qp-wormhole-circuit`	Leaf circuit definition, witness targets, and circuit fragments.
`qp-wormhole-prover`	Prover API for witness commitment and proof generation.
`qp-wormhole-verifier`	Lightweight verifier-facing interface.
`qp-wormhole-aggregator`	Recursive aggregation and wrapper logic.
`qp-wormhole-circuit-builder`	Serialization of prebuilt circuit artifacts for deployment.

Table 4: Repository-level implementation split.

8.2 Prover API

The prover uses a two-step interface: `commit` fills the witness, then `prove` generates the proof. This isolates witness assembly from proving for benchmarking.

8.3 Prebuilt binaries

The repository supports both prebuilt circuit binaries and in-memory reconstruction. Deployment uses prebuilt binaries for a stable artifact boundary.

9 Evaluation

We report wall-clock times for the current implementation.

9.1 Methodology

Benchmarks were executed on an Apple M2 Max with 12 CPU cores and 32 GB of RAM, running macOS 26.3.1 with `rustc 1.90.0` and `cargo 1.90.0`. To reduce interference from prior builds, runs used scratch-local `HOME`, `CARGO_HOME`, `TMPDIR`, and `CARGO_TARGET_DIR` rooted under `/tmp`.

For the implemented first aggregation layer, we evaluated same-`block_hash` batches with $N \in \{2, 4, 8, 9, 16, 25, 32, 36, 49\}$ leaf proofs. For the next recursive layer, we fixed each layer-0 proof to aggregate eight leaves and varied the number of input layer-0 proofs $M \in \{2, 4, 8, 16, 32\}$.

All first-layer measurements use repeated identical dummy leaf proofs, so they remain within the implemented same-block / shared-`block_hash` protocol surface; they do not measure heterogeneous real proofs drawn from a live block. The second-layer runs likewise use repeated dummy-derived layer-0 proofs with full batches only.

Reported build times are builder wall-clock times. For layer 0, build includes leaf and layer-0 artifact generation plus configuration hashing. For layer 1, build measures layer-1 artifact generation only, with prerequisite layer-0 artifacts generated separately. Reported prove times include backend load/deserialization and the `commit()` stage. Reported verify times isolate verification from proof generation, but still include verifier load/deserialization from serialized circuit binaries.

9.2 First-layer aggregation results

Across the tested range, first-layer build times lie between 7.46 s and 30.93 s, proving times between 3.72 s and 32.42 s, and verification times between 4.50 ms and 19.58 ms. Verification remains below 12 ms for all tested sizes up to $N = 36$, with the largest tested batch ($N = 49$) verifying in 19.58 ms. These results indicate that the refactored aggregation path preserves a very small verifier footprint even when the proving side grows into the tens-of-seconds regime.

N leaf proofs	Build (s)	Prove (s)	Verify (ms)
2	9.20	3.83	4.50
4	7.46	3.72	4.71
8	11.30	7.32	5.53
9	11.20	7.97	5.17
16	16.69	13.47	9.26
25	16.02	16.34	11.71
32	29.45	30.84	7.51
36	28.21	32.42	7.26
49	30.93	26.66	19.58

Table 5: Measured wall-clock times for the implemented first recursive aggregation layer. All runs use same-`block_hash` dummy leaf proofs and report the current refactored implementation only.

M layer-0 proofs	Leaves per layer-0 proof	Build (s)	Prove (s)	Verify (ms)
2	8	5.56	3.52	5.18
4	8	5.54	4.23	5.34
8	8	9.22	7.21	5.94
16	8	13.27	13.86	6.39
32	8	25.78	21.32	6.88

Table 6: Measured wall-clock times for the current second-layer aggregation interface. These measurements are head-only because the explicit public layer-1 surface is introduced by the refactor.

9.3 Second-layer aggregation results

For second-layer aggregation with eight leaves per input layer-0 proof, build times range from 5.54 s to 25.78 s, proving times from 3.52 s to 21.32 s, and verification times from 5.18 ms to 6.88 ms across $M \in \{2, 4, 8, 16, 32\}$. Verification is effectively flat across the tested fan-ins, while proving and artifact generation scale with the number of recursive inputs.

9.4 Forwarded payload and comparison basis

The wrapper-level public-output accounting is unchanged by the refactor. For a first-layer batch of size N, the forwarded semantic payload remains $14 + 8/N$ field elements per settled leaf, excluding zero padding in the wrapper's fixed-width public-input layout. This remains the correct accounting basis for any later throughput comparison against direct post-quantum signatures.

The current timing data establish the wall-clock behavior of the refactored aggregation flow.

10 Limitations

Same-block scoping. Layer-0 batches proofs sharing the same `block_hash`. Cross-block aggregation is future work.

Two-output limit. Each leaf supports two exits (spend + change), not general multi-recipient batching.

Amount quantization. Amounts are quantized to 0.01-unit resolution to reduce circuit cost.

Privacy leakage. Output amounts, exit addresses, and timing are public. Layer-0 padding enlarges the explanation class (section 7.8), but this is plausible deniability, not full shielding.

11 Related Work

Wormholes and privacy pools. EIP-7503 proposes proof-of-burn reminting via secret-derived addresses [6]. Privacy Pools adds compliance-aware privacy sets [3]. Quantus implements recursive proofs over authenticated chain state.

Shielded systems. Zcash provides strong privacy through shielded notes; Tornado Cash popularized mixer-style privacy on Ethereum [5, 10]. Quantus offers lighter-weight private settlement anchored in native chain state.

Recursive proofs. Halo and Plonky2 are standard recursive proof systems [2, 9]. Plonky3 provides Poseidon2 primitives with a separate recursion track [11, 12]. Quantus applies recursive composition to post-quantum private settlement using `qp-plonky2` with Poseidon2.

Post-quantum blockchains. ML-DSA adoption raises questions about transaction size and verification cost [7]. Quantus compresses authorization into ZK proofs rather than publishing large signatures on-chain.

12 Conclusion

Wormhole addresses let a post-quantum blockchain move authorization off-chain while preserving authenticated settlement. Users deposit to secret-derived addresses, prove the deposit exists and hasn't been spent, and reveal only settlement outputs. Recursive aggregation amortizes verification. We formalized the leaf relation, analyzed layer-0 privacy, proved security properties, and reported benchmark timings.

A Proof details and code linkage

This appendix links the main-text security arguments to implementation details.

A.1 Dummy-template implementation

The dummy template is serialized as `dummy_proof.bin` with all-zero outputs and `block_hash` $= 0^{32}$. The layer-0 prover pads batches by cloning this template, shuffles while preserving a real proof in slot 0, and generates random dummy nullifiers. The wrapper detects dummies via the sentinel `block_hash`.

A.2 Deposit binding

For a real leaf proof (non-zero `block_hash`), knowledge soundness extracts a witness satisfying $\mathcal{R}_{\text{leaf}}$: (C3) gives an authenticated storage proof, (C4) binds the header commitment, and the runtime checks the block exists in canonical history. This is the only step outside the circuit—the runtime validates the (`block_hash`, `block_number`) pair.

Claim used in the proofs	Primary file path	Concrete symbol / behavior
Designated dummy template exists as a serialized artifact	`wormhole/circuit-builder/src/lib.rs`	`generate_circuit_binaries` writes `dummy_proof.bin`.
Dummy witness uses zero block hash and zero outputs	`wormhole/aggregator/src/dummy_proof.rs`	`build_dummy_circuit_inputs`, `DUMMY_BLOCK_HASH`, `DUMMY_EXIT_ACCOUNT`.
Layer-0 padding clones exactly that template	`wormhole/aggregator/src/layer0/prover/lib.rs`	`dummy_proof_template.clone()` inside `commit`.
Real proof remains in slot 0 if any real proof exists	`wormhole/aggregator/src/layer0/prover/lib.rs`	`shuffle_proofs_preserving_first_real_layer0`.
Wrapper detects dummy children and substitutes filler nullifiers	`wormhole/aggregator/src/layer0/circuit/circuit_logic.rs`	`build_layer0_wrapper_constraints`; dummy flag from `block_hash` $= 0^{32}$.
Artifact integrity is checked against hashes stored with the run config	`wormhole/aggregator/src/config.rs`	`CircuitBinsConfig::verify_hashes`.
Layer-1 uses full batches and intentionally omits privacy padding	`wormhole/aggregator/src/aggregator.rs` `wormhole/aggregator/src/layer1/prover/lib.rs`	`Layer1Aggregator::aggregate` `Layer1AggregationProver::commit`.

Table 7: Code-level loci behind the security assumptions used in the appendix reductions.

A.3 Wrapper soundness

The wrapper verifies N child proofs and enforces: non-dummy children share `block_hash`, all children share `asset_id` and `volume_fee_bps` (from slot 0). Exit grouping sums amounts by destination and zeros duplicates. Nullifiers forward real values or dummy fillers based on the sentinel check.

Common `block_number` follows from leaf soundness: real children with the same `block_hash` have the same committed header, hence the same `block_number`.

A.4 Dummy-padding safety

Dummies have zero outputs and zero exit accounts, so they contribute nothing to forwarded amounts. The wrapper uses fresh random fillers for dummy nullifiers—this affects liveness (collision risk) not soundness.

A.5 Layer-0 ambiguity

Three sources enlarge the explanation class $\mathcal{E}(Y)$: (1) exit grouping hides individual contributions, (2) dummy nullifiers look random, (3) shuffle hides slot assignment. For grouped exits with total $S_a = m_a\Delta$ and multiplicity r_a, there are $\binom{m_a-1}{r_a-1}$ compositions—the stars-and-bars count from the main text.

A.6 One-time withdrawal

Two withdrawals from the same deposit record d would extract witnesses with the same counter c and wormhole address, hence $\mathsf{WA}(s) = \mathsf{WA}(s')$. Unless $s = s'$ (collision), both expose the same nullifier $\mathsf{Null}(s, c)$. The runtime rejects duplicates.

A.7 Value conservation

By wrapper soundness, aggregates expand into real children and dummies. Dummies contribute nothing. Each real child is bound to an authenticated deposit with a fee-constrained exit sum. Nullifier freshness prevents double-counting. Total settled value is bounded by total deposit value.

B Privacy-score heuristic

A wallet-side heuristic (not a formal guarantee): for candidate deposits $\mathcal{D}$ and withdrawal amount V,

$$\mathsf{Score}(V, \mathcal{D}) = \log_2 \left| \left\{ S \subseteq \mathcal{D} : \sum_{d \in S} d \cdot (1 - f) \approx V \right\} \right|.$$

C Exact public-input tables

For implementation-facing completeness, we preserve the exact leaf and wrapper layouts.

Index range	Field	Size (felts)
0	`asset_id`	1
1	`output_amount_1`	1
2	`output_amount_2`	1
3	`volume_fee_bps`	1
4–7	`nullifier`	4
8–11	`exit_account_1`	4
12–15	`exit_account_2`	4
16–19	`block_hash`	4
20	`block_number`	1

Table 8: Leaf public-input layout (21 felts total).

Offset	Field	Size
0	`num_exit_slots`	1
1	`asset_id`	1
2	`volume_fee_bps`	1
3–6	`block_hash`	4
7	`block_number`	1
8..	`[sum, exit_account]` repeated $2N$ times	5 per slot
next	`nullifier` repeated N times	4 per nullifier
rest	zero padding	until $21N + 8$ felts

Table 9: First-layer aggregate public-input layout.

References

[1] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP)*, 2018.

[2] Sean Bowe, Jack Grigg, and Daira Hopwood. Recursive Proof Composition without a Trusted Setup. Cryptology ePrint Archive, Report 2019/1021, 2019.

[3] Vitalik Buterin, Jacob Illum, Matthias Nadler, Fabian Sch"ar, and Ameen Soleimani. Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium. SSRN, 2023.

[4] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A Faster Version of the Poseidon Hash Function. Cryptology ePrint Archive, Report 2023/323, 2023.

[5] Daira Hopwood, Sean Bowe, Taylor Hornby, and Nathan Wilcox. Zcash Protocol Specification. Protocol specification, 2023.

[6] Keyvan Kambakhsh, Hamid Bateni, Amir Kahoori, and Nobitex Labs. EIP-7503: Zero-Knowledge Wormholes. Ethereum Improvement Proposal, 2023.

[7] National Institute of Standards and Technology. Module-Lattice-Based Digital Signature Standard (FIPS 204). Federal Information Processing Standards Publication 204, 2024.

[8] Parity Technologies. Substrate: The Blockchain Framework for a Multichain Future. Project documentation, 2023.

[9] Polygon Zero Team. Plonky2: Fast Recursive Arguments with PLONK and FRI. Software repository, 2022.

[10] Alexey Pertsev, Roman Semenov, and Roman Storm. Tornado Cash Privacy Solution. Project documentation, 2019.

[11] Plonky3 Contributors. Plonky3. Software repository, 2026.

[12] Plonky3 Contributors. Plonky3-recursion. Software repository, 2026.

[13] Quantus Network. qp-zk-circuits. Software repository, 2026.

[14] Quantus Network. qp-plonky2. Software repository, 2026.

[15] Peter W. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[16] Christopher Smith. QIP-0005: Entangled Addresses: Scalable Post-Quantum ZK-Signatures. Quantus Improvement Proposal draft, 2025.